# Lab 1 — Building a Mini Blockchain in Python

**Created by:** Mohamed Amine Marzouk **Email:** med.amine.mzk@horizon-tech.tn

**Course:** Blockchain: Concepts and Practice **Companion of:** `chapter-1-foundations-of-blockchain.pdf`
**Duration:** about 2 hours (40 min in class in Week 2 + homework) **Level:** 3rd-year engineering students, basic Python **Difficulty:** Beginner → Intermediate

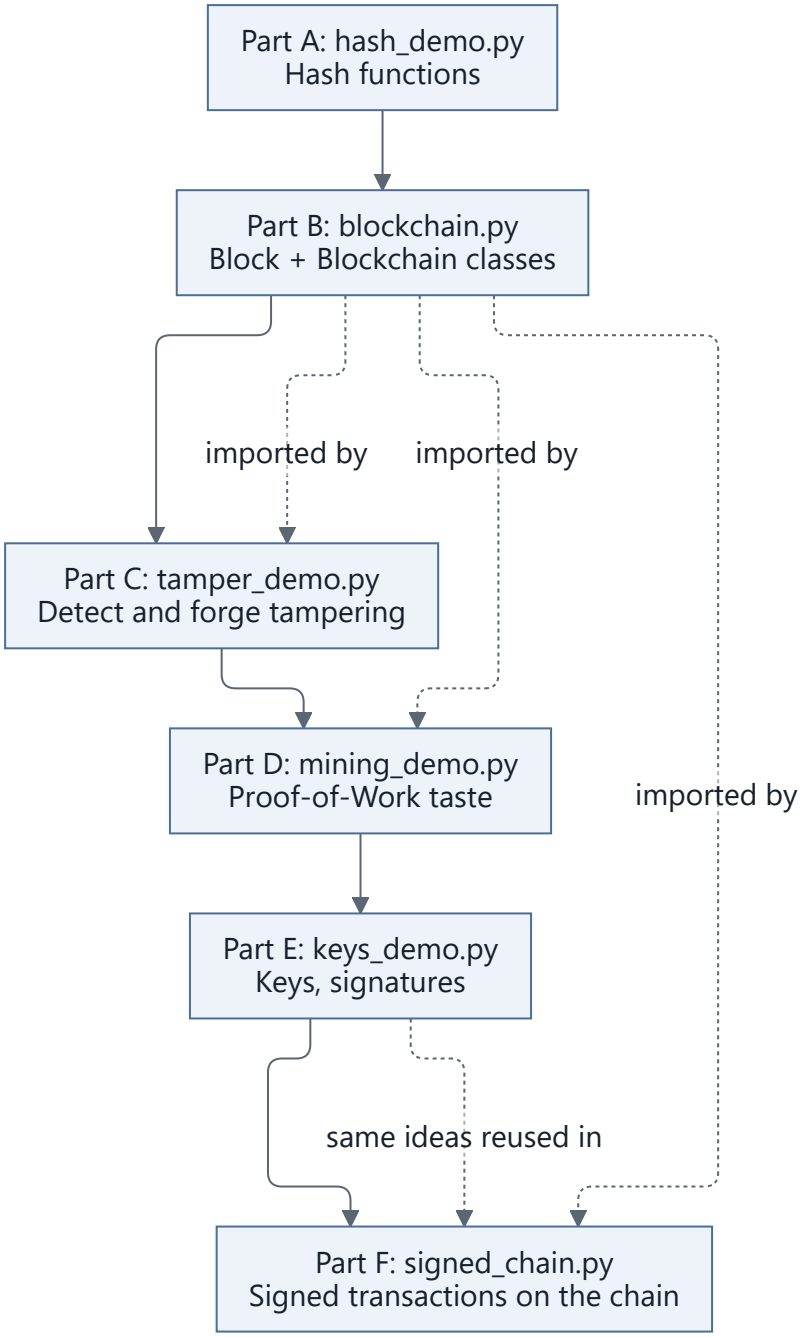
---

## 1. Objectives

By the end of this lab, students will have:

1. Computed and compared SHA-256 hashes in Python.
2. Implemented a `Block` and a `Blockchain` class with hash linking.
3. Written a chain **validator** and used it to **detect tampering**.
4. Experienced a **Proof-of-Work** style mining loop and measured how difficulty changes the effort (a preview of Chapter 2).
5. Generated **ECDSA key pairs**, signed and verified messages.
6. Combined everything: a chain whose transactions are **signed** and **verified** before being accepted.

## 2. Lab architecture




## 3. Prerequisites and setup

- Python 3.9 or newer ( `python3 --version` )
- A terminal and a text editor (VS Code recommended)
- Internet access once, to install the `ecdsa` package (used only in Parts E and F)

Create a working folder and a virtual environment:

```
mkdir blockchain-lab1
cd blockchain-lab1
python3 -m venv venv

# Linux / macOS
source venv/bin/activate
# Windows (PowerShell)
# venv\Scripts\Activate.ps1

pip install ecdsa
```

> **No internet?** Parts A–D need only the Python standard library. Parts E and F need the `ecdsa` package; you can pre-download the wheel with `pip download ecdsa six` and install it from a USB drive or a local server.

---

## Part A — Hash functions (15 min)

**Goal:** see determinism, the avalanche effect and fixed-size output with your own eyes.

Create `hash_demo.py`:

```
# file: hash_demo.py
import hashlib


def sha256_hex(text: str) -> str:
    """Return the SHA-256 digest of a string as 64 hex characters."""
    return hashlib.sha256(text.encode("utf-8")).hexdigest()


def differing_bits(hex_a: str, hex_b: str) -> int:
    """Count how many of the 256 bits differ between two digests."""
    return bin(int(hex_a, 16) ^ int(hex_b, 16)).count("1")


if __name__ == "__main__":
    samples = ["hello", "Hello", "hello!", "", "Alice pays Bob 5 coins"]
    for s in samples:
        print(f"{s!r:30} -> {sha256_hex(s)}")

    print()
    # 1) Determinism
    print("Same input twice, same hash?",
          sha256_hex("hello") == sha256_hex("hello"))

    # 2) Fixed size, whatever the input
    long_text = "blockchain " * 100_000
    print("Digest length for a 1.1 MB input:", len(sha256_hex(long_text)), "hex chars")

    # 3) Avalanche effect
    a, b = sha256_hex("hello"), sha256_hex("Hello")
    print(f"Bits that differ between 'hello' and 'Hello': {differing_bits(a, b)} / 256")
```

Run it:

```
python hash_demo.py
```

**Expected output (first lines):**

```
'hello'                       -> 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
'Hello'                       -> 185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969
...
Same input twice, same hash? True
Digest length for a 1.1 MB input: 64 hex chars
Bits that differ between 'hello' and 'Hello': 125 / 256
```

**Checkpoint questions**

1. Why is the digest always 64 characters long?
2. About how many bits should differ after a one-letter change if the avalanche effect holds? (Answer: about half, i.e. around 128.)
3. Try five more pairs of strings that differ by one character. What range of differing bits do you observe?

---

## Part B — The Block and Blockchain classes (30 min)

**Goal:** implement the data structure of Section 7 of the chapter.

Design decisions (read these before you code):

- A block's **hash covers all its fields** — index, timestamp, transactions, previous hash and nonce. Change any of them and the hash changes.
- We serialize the fields with `json.dumps(..., sort_keys=True)` so that the same content always gives the same bytes (**determinism matters!**).
- The **genesis block** has `previous_hash = "0" * 64` and a fixed timestamp, so every run starts with the same first block.
- We include a `nonce` and a `mine()` method already; with `difficulty = 0` they have no effect until Part D.

Create `blockchain.py`:

```python
# file: blockchain.py
import hashlib
import json
import time


class Block:
    def __init__(self, index, transactions, previous_hash, timestamp=None, nonce=0):
        self.index = index
        self.timestamp = timestamp if timestamp is not None else time.time()
        self.transactions = transactions      # a list (JSON-serializable)
        self.previous_hash = previous_hash
        self.nonce = nonce
        self.hash = self.compute_hash()

    def compute_hash(self) -> str:
        """SHA-256 of every field of the block, in a stable order."""
        payload = json.dumps(
            {
                "index": self.index,
                "timestamp": self.timestamp,
                "transactions": self.transactions,
                "previous_hash": self.previous_hash,
                "nonce": self.nonce,
            },
            sort_keys=True,
        )
        return hashlib.sha256(payload.encode("utf-8")).hexdigest()

    def mine(self, difficulty: int) -> None:
        """Increase the nonce until the hash starts with `difficulty` zeros."""
        target = "0" * difficulty
        while not self.hash.startswith(target):
            self.nonce += 1
            self.hash = self.compute_hash()

    def __repr__(self) -> str:
        return (f"Block(index={self.index}, prev={self.previous_hash[:8]}..., "
                f"hash={self.hash[:8]}..., nonce={self.nonce}, "
                f"txs={len(self.transactions)})")


class Blockchain:
    def __init__(self, difficulty: int = 0):
        self.difficulty = difficulty
        self.chain = [self._create_genesis_block()]

    def _create_genesis_block(self) -> Block:
        genesis = Block(0, ["Genesis block"], "0" * 64, timestamp=0)
        genesis.mine(self.difficulty)
        return genesis

    @property
    def last_block(self) -> Block:
        return self.chain[-1]

    def add_block(self, transactions) -> Block:
        block = Block(len(self.chain), transactions, self.last_block.hash)
        block.mine(self.difficulty)
        self.chain.append(block)
        return block
```

```
    def is_valid(self):
        """Return (True, message) if the chain is consistent, else (False, reason)."""
        prefix = "0" * self.difficulty
        for i, block in enumerate(self.chain):
            if block.hash != block.compute_hash():
                return False, f"Block {i}: stored hash does not match its content"
            if not block.hash.startswith(prefix):
                return False, f"Block {i}: hash does not satisfy the difficulty"
            if i == 0:
                if block.previous_hash != "0" * 64:
                    return False, "Genesis block has a wrong previous hash"
            elif block.previous_hash != self.chain[i - 1].hash:
                return False, f"Block {i}: link to block {i - 1} is broken"
        return True, "Chain is valid"

    def print_chain(self) -> None:
        for block in self.chain:
            print(block)
            for tx in block.transactions:
                print("    ", tx)


if __name__ == "__main__":
    bc = Blockchain()
    bc.add_block(["Alice pays Bob 5"])
    bc.add_block(["Bob pays Carol 2", "Carol pays Dave 1"])
    bc.print_chain()
    print(bc.is_valid())
```

Run it:

```
python blockchain.py
```

**Expected output (hashes and nonces differ on your machine because of timestamps):**

```
Block(index=0, prev=00000000..., hash=........, nonce=0, txs=1)
     Genesis block
Block(index=1, prev=........, hash=........, nonce=0, txs=1)
     Alice pays Bob 5
Block(index=2, prev=........, hash=........, nonce=0, txs=2)
     Bob pays Carol 2
     Carol pays Dave 1
(True, 'Chain is valid')
```

### Checkpoint questions

1. Which line of the code creates the "chain" property? (Answer: `Block(len(self.chain), transactions, self.last_block.hash)` — each block receives the previous block's hash.)
2. Why do we sort the JSON keys? What would happen if we did not? (Dictionaries might be serialized in a different order on another machine → different hash → nodes would disagree.)
3. Why is the genesis block's timestamp fixed to 0 and not `time.time()`? (So that every node creates exactly the same genesis block.)

---

## Part C — Tampering and detection (20 min)

**Goal:** reproduce the scenario of Section 7.4 of the chapter, and understand *why a hash chain alone is not enough*.

Create `tamper_demo.py`:

```
# file: tamper_demo.py
import sys
import time

from blockchain import Blockchain


def forge_from(bc: Blockchain, start: int) -> None:
    """Attacker recomputes (and re-mines) every block from `start` onward."""
    for i in range(start, len(bc.chain)):
        block = bc.chain[i]
        block.previous_hash = bc.chain[i - 1].hash
        block.nonce = 0
        block.hash = block.compute_hash()
        block.mine(bc.difficulty)


if __name__ == "__main__":
    difficulty = int(sys.argv[1]) if len(sys.argv) > 1 else 0

    bc = Blockchain(difficulty=difficulty)
    bc.add_block(["Alice pays Bob 5"])
    bc.add_block(["Bob pays Carol 2"])
    bc.add_block(["Carol pays Dave 1"])

    print("1) Honest chain       :", bc.is_valid())

    # --- Attack 1: change data only ---------------------------------------
    bc.chain[1].transactions = ["Alice pays Bob 50"]
    print("2) Data edited        :", bc.is_valid())

    # --- Attack 2: also recompute the hash of the edited block ------------
    bc.chain[1].hash = bc.chain[1].compute_hash()
    bc.chain[1].mine(bc.difficulty)
    print("3) Block 1 re-hashed  :", bc.is_valid())

    # --- Attack 3: recompute every later block too ------------------------
    start = time.time()
    forge_from(bc, 1)
    elapsed = time.time() - start
    print("4) Whole tail forged  :", bc.is_valid(), f"(took {elapsed:.3f} s)")
```

Run it twice, first with no mining and then with difficulty 4:

```
python tamper_demo.py
python tamper_demo.py 4
```

**Expected output with difficulty 0:**

```
1) Honest chain       : (True, 'Chain is valid')
2) Data edited        : (False, 'Block 1: stored hash does not match its content')
3) Block 1 re-hashed  : (False, 'Block 2: link to block 1 is broken')
4) Whole tail forged  : (True, 'Chain is valid') (took 0.000 s)
```

**With difficulty 4:** the result is the same, but step 4 takes noticeably longer (typically a fraction of a second to a few seconds) because the attacker must **re-mine** three blocks.

**Discussion points (important!)**

1. Steps 2 and 3 show that the chain is **tamper-evident**: every edit is *detected* by `is_valid()`.
2. Step 4 shows the limit: a lone attacker who controls the *only* copy can forge the whole chain. A real network has **thousands of independent copies** and the work to forge must be redone faster than the honest network adds new blocks (Chapter 2).
3. The higher the difficulty, the higher the attacker's cost. This is the entire economic idea behind Proof of Work.

---

## Part D — A taste of Proof of Work (15 min)

**Goal:** measure how mining effort grows with difficulty.

Create `mining_demo.py`:

```python
# file: mining_demo.py
import time

from blockchain import Block

if __name__ == "__main__":
    print(f"{'difficulty':>10} {'nonce':>10} {'time (s)':>10}  hash")
    for difficulty in range(1, 6):
        block = Block(1, ["Alice pays Bob 5"], "0" * 64, timestamp=0)
        start = time.time()
        block.mine(difficulty)
        elapsed = time.time() - start
        print(f"{difficulty:>10} {block.nonce:>10} {elapsed:>10.3f}  {block.hash[:24]}...")
```

Run it:

```
python mining_demo.py
```

**Expected output (the nonces are deterministic because the timestamp is fixed; times depend on your CPU, and on the test machine difficulty 5 needed nonce 681875 and about 5 seconds):**

```
difficulty      nonce   time (s)  hash
         1          ...      0.000  0...
         2          ...      0.000  00...
         3          ...      0.0xx  000...
         4          ...      0.xxx  0000...
         5          ...      x.xxx  00000...
```

**Questions**

1. Each extra zero hex character multiplies the expected number of attempts by what factor? (Answer: 16, because each hex character has 16 possible values.)
2. Using your measured time at difficulty 5, estimate the time for difficulty 8. (Roughly × $16^3$ = 4,096.) Why shouldn't we run it?
3. In recent years, Bitcoin block hashes have started with roughly 19–20 zero hex characters (look at any recent block on a block explorer). What does this tell you about the hardware miners use?
4. Verification takes **one hash**; finding takes **millions**. Why is that asymmetry essential?

> **Do not run difficulty 6 or above in Python.** It can take minutes. Use it instead to discuss why miners moved from CPUs to GPUs to specialised ASIC hardware.

---

## Part E — Keys and digital signatures (20 min)

**Goal:** generate a key pair, sign a message, verify it, and see what happens when the message is changed.

> `hashfunc=hashlib.sha256` is passed explicitly because the `ecdsa` library defaults to SHA-1 otherwise.

Create `keys_demo.py`:

```python
# file: keys_demo.py
import hashlib

from ecdsa import SECP256k1, BadSignatureError, SigningKey

if __name__ == "__main__":
    # 1) Generate a key pair on the same curve as Bitcoin and Ethereum
    private_key = SigningKey.generate(curve=SECP256k1)
    public_key = private_key.get_verifying_key()

    print("Private key (SECRET!):", private_key.to_string().hex())
    print("Public key           :", public_key.to_string().hex())

    # 2) A simplified address: first 40 hex chars of SHA-256(public key)
    address = hashlib.sha256(public_key.to_string()).hexdigest()[:40]
    print("Address (simplified) :", address)

    # 3) Sign a message
    message = b"Alice pays Bob 5 coins"
    signature = private_key.sign(message, hashfunc=hashlib.sha256)
    print("\nSignature            :", signature.hex()[:64], "...")

    # 4) Verify the genuine message
    print("Genuine message valid?", public_key.verify(signature, message, hashfunc=hashlib.sha256))

    # 5) Verify a modified message
    tampered = b"Alice pays Bob 500 coins"
    try:
        public_key.verify(signature, tampered, hashfunc=hashlib.sha256)
        print("Tampered message valid? True  (this should NEVER happen)")
    except BadSignatureError:
        print("Tampered message valid? False (signature rejected)")

    # 6) Verify with somebody else's public key
    other_public = SigningKey.generate(curve=SECP256k1).get_verifying_key()
    try:
        other_public.verify(signature, message, hashfunc=hashlib.sha256)
        print("Wrong public key valid? True  (this should NEVER happen)")
    except BadSignatureError:
        print("Wrong public key valid? False (not signed by that key)")
```

Run it:

```
python keys_demo.py
```

**Expected output (keys and signature differ on each run; the verdicts are fixed):**

```
Private key (SECRET!): ........
Public key           : ........
Address (simplified) : ........

Signature            : ........ ...
Genuine message valid? True
Tampered message valid? False (signature rejected)
Wrong public key valid? False (not signed by that key)
```

### Checkpoint questions

1. Run the script twice. Why are the keys different each time?

2. Which key is used to *sign*? Which to *verify*?
3. Which of the three guarantees (authentication, integrity, non-repudiation) is demonstrated by step 5? by step 6?
4. The private key is only 32 bytes. Where would you store it in a real application? (Hardware wallet, OS keystore, encrypted file, never in source code or Git.)

> **Security reminder:** never reuse these demo keys for anything real. Never paste a private key into a website, chat or repository.

---

## Part F — Putting it all together: a chain of signed transactions (30 min)

**Goal:** accept only transactions that are correctly signed by the owner of the *sender address*.

Rules implemented by the verifier:

1. The `public_key` in the transaction must hash to the claimed `sender` address — otherwise Mallory could sign with *her* key but write *Alice's* address.
2. The signature must be valid for the content `(sender, receiver, amount)`.
3. The amount must be positive.

> Balance tracking and double-spend detection are left as a challenge (Section 5) so you can practise extending the design.

Create `signed_chain.py`:

```python
# file: signed_chain.py
import hashlib
import json

from ecdsa import SECP256k1, BadSignatureError, SigningKey, VerifyingKey

from blockchain import Blockchain


def address_from_public_key(public_key_bytes: bytes) -> str:
    return hashlib.sha256(public_key_bytes).hexdigest()[:40]


def signable_bytes(tx: dict) -> bytes:
    """The exact bytes that are signed: sender, receiver, amount."""
    content = {k: tx[k] for k in ("sender", "receiver", "amount")}
    return json.dumps(content, sort_keys=True).encode("utf-8")


class Wallet:
    def __init__(self, name: str):
        self.name = name
        self._private_key = SigningKey.generate(curve=SECP256k1)
        public_key = self._private_key.get_verifying_key()
        self.public_key_hex = public_key.to_string().hex()
        self.address = address_from_public_key(public_key.to_string())

    def create_transaction(self, receiver_address: str, amount: float) -> dict:
        tx = {
            "sender": self.address,
            "receiver": receiver_address,
            "amount": amount,
            "public_key": self.public_key_hex,
        }
        tx["signature"] = self._private_key.sign(
            signable_bytes(tx), hashfunc=hashlib.sha256
        ).hex()
        return tx


def verify_transaction(tx: dict) -> bool:
    try:
        if tx["amount"] <= 0:
            return False
        public_key_bytes = bytes.fromhex(tx["public_key"])
        # Rule 1: the key must match the claimed sender address
        if address_from_public_key(public_key_bytes) != tx["sender"]:
            return False
        # Rule 2: the signature must be valid for this content
        verifying_key = VerifyingKey.from_string(public_key_bytes, curve=SECP256k1)
        return verifying_key.verify(
            bytes.fromhex(tx["signature"]), signable_bytes(tx), hashfunc=hashlib.sha256
        )
    except (BadSignatureError, KeyError, ValueError):
        return False


def add_transactions(bc: Blockchain, transactions: list) -> None:
    """Add a block containing only the valid transactions."""
    accepted = []
    for tx in transactions:
```

```
        ok = verify_transaction(tx)
        print(f"  tx {tx['amount']:>6} : {'ACCEPTED' if ok else 'REJECTED'}")
        if ok:
            accepted.append(tx)
    if accepted:
        bc.add_block(accepted)


if __name__ == "__main__":
    alice, bob, mallory = Wallet("Alice"), Wallet("Bob"), Wallet("Mallory")
    bc = Blockchain(difficulty=3)

    print("Block 1: honest payment")
    add_transactions(bc, [alice.create_transaction(bob.address, 5)])

    print("Block 2: three attempts")
    honest = bob.create_transaction(alice.address, 2)

    modified = alice.create_transaction(bob.address, 5)
    modified["amount"] = 500                    # amount changed after signing

    forged = mallory.create_transaction(bob.address, 10)
    forged["sender"] = alice.address            # Mallory pretends to be Alice

    add_transactions(bc, [honest, modified, forged])

    print()
    bc.print_chain()
    print("Chain valid?", bc.is_valid())
```

Run it:

```
python signed_chain.py
```

**Expected output (hashes shortened and differ per run; verdicts are fixed):**

```
Block 1: honest payment
  tx      5 : ACCEPTED
Block 2: three attempts
  tx      2 : ACCEPTED
  tx    500 : REJECTED
  tx     10 : REJECTED

Block(index=0, ...)
...
Chain valid? (True, 'Chain is valid')
```

### Questions

1. Why is the `modified` transaction rejected? Which cryptographic property protects against it?
2. Why is the `forged` transaction rejected? Which rule catches it?
3. Where in the code would you add a **balance check**? What data would you need? (A function that replays the chain and computes each address's balance.)
4. Someone *replays* an accepted transaction in a later block. Does the current code detect it? (No — see Challenge 3.)

---

## 4. Troubleshooting

| Problem | Likely cause | Fix |
|---|---|---|
| `ModuleNotFoundError: No module named 'ecdsa'` | Virtual environment not active or package not installed | Activate the venv, run `pip install ecdsa` |
| `ModuleNotFoundError: No module named 'blockchain'` | Running a script from another folder | `cd` into the lab folder; all files must be in the same directory |
| Hash differs from the table in the chapter | Extra newline or different encoding | Use `echo -n` in a shell; `.encode("utf-8")` in Python |
| `is_valid()` is `False` right after adding blocks | Changed a block field after hashing without recomputing | Call `compute_hash()` (or `mine`) after modifying any field |
| Mining never ends | Difficulty too high for Python | Use difficulty 1–5 only |

---

## 5. Challenges (bonus, homework)

### Challenge 1 — Balances

Add a `get_balance(address)` method that replays the chain. Give each wallet an initial balance via a special `"coinbase"` transaction in a first block, and reject transactions that overspend.

### Challenge 2 — Block explorer

Write a small CLI that prints a block by index and prints the full history of an address.

### Challenge 3 — Replay protection

Add a `nonce` (or a counter) per sender in each transaction, include it in the signed bytes, and reject any transaction whose number was already used. Explain why this is needed (a valid signed message is valid *forever*).

### Challenge 4 — Persistence

Save the chain to `chain.json` and reload it with full validation on load. What must you recompute and what must you check?

### Challenge 5 — Two nodes

Run two instances of the chain and write a function that picks the longest valid chain. (This is a first approximation of what nodes do in Chapter 2.)

### Challenge 6 — Merkle root (preview of Chapter 2)

Compute a single hash that summarizes all the transactions of a block by hashing pairs repeatedly until one hash remains.

---

## 6. Deliverables (to submit)

A single folder or ZIP containing:

1. The six Python files, working.
2. A short report (1–2 pages, PDF or Markdown) containing:
   - The output of each part (screenshots or pasted text).
   - Answers to **all** the checkpoint questions of Parts A–F.
   - A paragraph titled *"Why a hash chain alone is not secure, and what is missing"* (Part C discussion).
   - At least one challenge attempted (and a sentence on what you learned).

### Suggested grading rubric (20 points)

| Criterion | Points |
|---|---|
| Parts A–B working (hashing, block, chain, validation) | 5 |
| Part C: tamper demo working and correctly explained | 4 |
| Part D: mining table and correct answers to the questions | 3 |
| Parts E–F: signatures and signed transactions working | 4 |
| Report quality: clear, complete, honest about limitations | 2 |
| Challenge attempted (bonus up to +2) | +2 |

▸ **Teacher's guide** — preparation, pacing and answers